

Тестовое задание: Middle Backend Developer

- Стек: PHP 8.1+ или PHP 8.2+, Laravel
- База данных: на выбор кандидата, если нужна
- Формат сдачи: ссылка на репозиторий или архив с проектом

Задача

Нужно реализовать модуль работы с несколькими платёжными системами. В системе есть две платёжные системы:

- PayGateA
- PayGateB

У каждой платёжной системы есть два сценария:

1. создание платежа,
2. обработка callback от провайдера.

Нужно сделать единый backend API, через который приложение работает с обеими платёжными системами, несмотря на различия в их форматах.

Функциональные требования

1. Создание платежа

Реализовать endpoint: `POST /api/payments`

Пример запроса:

```
JSON
{
  "provider": "paygate_a",
  "amount": "100.00",
  "currency": "USD",
  "order_id": "ORD-10001",
  "description": "Test payment"
}
```

`order_id` - на ваш выбор, можете создать бд и обработать внутри или использовать как в примере

Что ожидается:

- endpoint принимает запрос на создание платежа
- в зависимости от `provider` использует нужную платёжную систему

- сохраняет информацию о платеже локально (можно не хранить если не разворачиваете бд)
- возвращает клиенту нормализованный ответ

Пример нормализованного ответа:

JSON

```
{
  "id": 1,
  "provider": "paygate_a",
  "external_id": "ext-50001",
  "status": "pending",
  "payment_url": "https://example.com/pay/abc123"
}
```

2. Callback от платёжной системы

Реализовать endpoint: `POST /api/callbacks/{provider}`

Где `{provider}` может быть, например:

- `paygate-a`
- `paygate-b`

Что ожидается:

- каждая платежная система присылает callback в своём формате
- приложение должно определить нужный обработчик по `provider`
- callback должен находить нужный локальный платёж
- внутренний статус платежа должен обновляться в нормализованный статус системы

Минимально ожидаемые внутренние статусы:

- `pending`
- `success`
- `failed`

Контракты провайдеров

Ниже специально даны разные форматы, чтобы проверить, как кандидат отделяет интеграционную логику от общей.

PayGateA

Создание платежа

`POST /external/paygate-a/create`

Запрос:

JavaScript

```
{
  "amount": "100.00",
  "currency": "USD",
  "merchant_order_id": "ORD-10001"
}
```

Ответ:

JSON

```
{
  "payment_id": "a-100500",
  "payment_url": "https://paygate-a.test/pay/a-100500",
  "status": "new"
}
```

Callback: `POST /api/callbacks/{provider}` при `provider=paygate-a`

Payload:

JSON

```
{
  "payment_id": "a-100500",
  "merchant_order_id": "ORD-10001",
  "status": "paid"
}
```

Mapping статусов:

- `new` -> `pending`
- `paid` -> `success`
- `rejected` -> `failed`

PayGateB

Создание платежа

POST /external/paygate-b/payments

Запрос:

```
JSON
{
  "order": "ORD-10001",
  "total": 10000,
  "currency_code": "USD",
  "note": "Test payment"
}
```

Примечание: `total` передаётся в центах.

Ответ:

```
JSON
{
  "id": "b-777",
  "redirect_url": "https://paygate-b.test/checkout/b-777",
  "state": "created"
}
```

Callback: POST /api/callbacks/{provider} при provider=paygate-{b|a}

Payload:

```
JSON
{
  "id": "b-777",
  "order": "ORD-10001",
  "state": "success"
}
```

Mapping статусов:

- ``created`` -> ``pending``

- ``success`` -> ``success``
- ``error`` -> ``failed``

Нефункциональные ожидания

Важно чтобы решение было расширяемым на N провайдеров.

Ожидается, что:

- логика выбора провайдера не размазана по controller
- код не дублируется между платежными системами без необходимости
- provider-specific части изолированы
- добавление третьей платежной системы не требует переписывания основного flow

При этом не требуется чрезмерно сложная архитектура.

Ограничения

- не требуется frontend
- не требуется реальная интеграция с настоящими платежками
- можно реализовать mock/fake провайдеры внутри проекта
- БД не обязательно, если нужна то любая

Что нужно сдать

1. Исходный код
2. `README` с инструкцией по запуску
3. Краткое описание решения:
 - как организована структура,
 - как добавлять нового provider,
 - почему были выбраны именно такие абстракции

Что будет плюсом

- внятный `PaymentProviderInterface` или аналогичная уместная абстракция
- отдельные mapper / adapter классы
- аккуратные DTO, если они действительно упрощают код
- базовая валидация callback